

# FaceVault: A Dual-Model Real-Time Face Recognition System with Dynamic Enrollment and Open-Set Unknown Detection

Hareekrishna Vellore Shanmugam, Hariharan Duraisingh, and Harshavardhan Anbuchezhian Gowri

Concordia University, Montreal, QC, Canada

ID: 40329089, 40303001, 40323293

<https://github.com/hareekrishnavs/Face-Recognition-Application>

**Abstract**—We present *FaceVault*, a real-time dual-model face recognition web application that integrates a custom Convolutional Neural Network (CNN) classifier with InsightFace ArcFace-based embedding matching. The CNN is trained from scratch on a small private dataset of six enrolled team members (178 raw images), augmented to 1370 training samples using nine geometric and photometric transforms. A two-stage decision rule—combining a softmax confidence floor ( $\tau_{\text{conf}} = 0.655$ ) and a margin gap ( $\tau_{\text{margin}} = 0.207$ )—separates known identities from unknown visitors. The InsightFace module provides complementary open-set detection via cosine similarity in a 512-dimensional embedding space, enabling background-thread enrollment of new identities without restarting the server. The CNN achieves 88.89% validation accuracy and 83.33% test accuracy across the six-class task; InsightFace achieves 94.44% validation accuracy and 88.89% test accuracy using the enrolled prototypes. The complete system is delivered as a Flask + Socket.IO single-page application with live alignment guidance, blur detection, multi-frame stability consensus, real-time activity logging, and a runtime threshold slider.

**Index Terms**—face recognition, CNN, ArcFace, InsightFace, open-set detection, dynamic enrollment, Flask, Socket.IO, real-time system

## I. INTRODUCTION

Face recognition has evolved into a core component of modern access control, attendance tracking, and personalized human-computer interaction. The central challenge in real deployment is the *open-set* problem: a system must correctly label enrolled individuals while flagging all others as *Unknown*, rather than forcing every face into one of the training classes.

Two dominant paradigms address this. Closed-set *classifiers* based on softmax cross-entropy produce well-calibrated probability scores suitable for confidence thresholding. Metric-learning objectives such as ArcFace [1] project face images into a hyperspherical embedding space where cosine distance measures identity similarity without committing to a fixed class set, enabling zero-shot enrollment.

*FaceVault* unifies both paradigms. A lightweight custom CNN handles fast, interpretable classification for enrolled identities; the InsightFace [2] `buffalo_l` backbone provides ArcFace embeddings for dynamic enrollment and open-set rejection. The active model can be switched at runtime without restarting the server.

## Contributions:

- A four-block CNN with dual-threshold unknown detection, trained on a small private dataset with nine-type augmentation.
- An InsightFace integration with prototype-blended cosine scoring for open-set detection and background-thread enrollment.
- A multi-frame stability consensus pipeline (IoU tracking, blur detection, alignment guidance) for robust real-world inference.
- A Flask + Socket.IO dashboard with live activity logging, CSV export, and runtime threshold control.

## II. RELATED WORK

FaceNet [3] introduced triplet-loss training on 128-dimensional embeddings, establishing the metric-learning paradigm for face verification. ArcFace [1] improved separation by adding an additive angular margin to the softmax loss, achieving state-of-the-art accuracy on LFW and IJB-C benchmarks. InsightFace [2] packages ArcFace with RetinaFace [4] detection and makes them available as a ready-to-use inference library.

For open-set recognition, threshold-based softmax rejection [7] is the simplest approach; more principled methods include OpenMax [6] and nearest-prototype distance in embedding space. *FaceVault* employs threshold-based rejection for both models, with an additional margin check for the CNN.

Haar Cascade detectors [5] provide fast CPU-based face detection; RetinaFace [4] offers superior accuracy under pose variation. *FaceVault* uses Haar Cascades in the CNN pipeline and RetinaFace inside the InsightFace pipeline to balance speed with accuracy.

## III. DATASET COLLECTION

### A. Collection Protocol

The dataset comprises **178 raw images** of six team members captured using smartphone cameras under varied indoor lighting conditions, at multiple distances and head-pose angles (frontal, slight left/right profile). No controlled studio equipment was used, reflecting a realistic self-enrollment scenario. Images were submitted in JPEG, PNG, and HEIC formats.



Fig. 1: Sample raw images from all six enrolled identities (Haree, HariHaran, Harsha, Jacob, Jousha, Akhil). Natural variation in lighting, pose, and background is deliberately retained.

The directory structure follows one subfolder per identity:

```
dataset/raw/
Akhil/      (30 images)
Haree/      (30 images)
HariHaran/  (31 images)
Harsha/     (34 images)
Jacob/      (16 images)
Jousha/     (37 images)
```

## B. Dataset Statistics

Table I summarizes per-identity image counts. The class imbalance (16 images for Jacob vs. 37 for Jousha) reflects natural variation in availability of photographs. Harsha lost 3 images and Jacob 1 image to failed face detection, giving 173 usable crops from 178 raw images.

TABLE I: Raw Image Counts per Identity

| Identity     | Raw Images | After Face Detection |
|--------------|------------|----------------------|
| Akhil        | 30         | 29                   |
| Haree        | 30         | 30                   |
| HariHaran    | 31         | 31                   |
| Harsha       | 34         | 31                   |
| Jacob        | 16         | 15                   |
| Jousha       | 37         | 37                   |
| <b>Total</b> | <b>178</b> | <b>173</b>           |

## IV. DATA PREPROCESSING

### A. Formatting and EXIF Correction

All images are loaded with Pillow. HEIC/HEIF formats are handled via the `pillow_heif` plugin. EXIF orientation metadata is applied before any processing to ensure a consistent upright orientation regardless of the capture device. All output images are saved as JPEG for uniform pipeline compatibility.

### B. Face Detection and Cropping

Each image undergoes the following detection pipeline:

- 1) Convert to grayscale and apply histogram equalization to improve contrast.
- 2) Run OpenCV Haar Cascade face detection (frontal, alt, profile cascades).
- 3) If no face is found after equalization, retry on the original grayscale image.
- 4) If still no face is detected, discard the image.

For each detected face bounding box:

- A **28% margin** is added on all sides to retain hair, jawline, and ear context.
- The expanded region is center-cropped and resized to  $160 \times 160$  pixels.

Images where no face is detected are discarded (5 of 178 raw images). A minimum of three valid face crops per class is required; otherwise, training is aborted.

### C. Train / Validation / Test Split

The face-cropped images are partitioned using **stratified** sampling:

- **80%** Training
- **10%** Validation
- **10%** Test

A fixed random seed of 42 ensures reproducibility. Splitting is performed *before* augmentation to prevent data leakage: if augmented versions of a training image appeared in the validation or test set, performance would be artificially inflated.

### D. Data Augmentation

Augmentation is applied **only to the training split**. For each original training image, nine augmented variants are generated:

- 1) **Rotate Left** ( $-12^\circ$ )
- 2) **Rotate Right** ( $+12^\circ$ )
- 3) **Scale In** ( $+8\%$  zoom)
- 4) **Scale Out** ( $-8\%$  zoom)
- 5) **Gaussian Noise** ( $\sigma = 8.0$ )
- 6) **Gaussian Blur** (radius = 1.1)
- 7) **Brightness** (factor  $\times 1.16$ )
- 8) **Contrast** (factor  $\times 1.18$ )
- 9) **Horizontal Flip**



Fig. 2: Nine augmented variants of a single training image (HariHaran). Left-to-right: original, rotate-left, rotate-right, scale-in, scale-out, Gaussian noise, Gaussian blur, brightness, contrast.

Table II shows the final split sizes after augmentation. The training set grows by a factor of  $10\times$  (original + nine augmented variants).

TABLE II: Dataset Split Sizes After Augmentation

| Identity     | Train       | Val       | Test      | Total       |
|--------------|-------------|-----------|-----------|-------------|
| Akhil        | 230         | 3         | 3         | 236         |
| Haree        | 240         | 3         | 3         | 246         |
| HariHaran    | 250         | 3         | 3         | 256         |
| Harsha       | 250         | 3         | 3         | 256         |
| Jacob        | 110         | 2         | 2         | 114         |
| Jousha       | 290         | 4         | 4         | 298         |
| <b>Total</b> | <b>1370</b> | <b>18</b> | <b>18</b> | <b>1406</b> |

## V. MODEL ARCHITECTURE AND TRAINING

Our final system uses two complementary models: a Custom CNN and InsightFace. Several architectures were explored before settling on this combination.

### A. Preliminary Experiments

We initially experimented with a **ResNet-50** backbone for both embedding-based and image-based classification. While the model achieved near-100% training accuracy, it suffered from severe overfitting on the small private dataset and performed poorly in real-world webcam testing. The large parameter count of ResNet-50 (~25M) far exceeded what our dataset could support, leading to a large generalization gap.

We subsequently tested several lightweight custom CNN architectures. The selected design (described below) provided the best balance between performance and generalization, achieving 100% test accuracy with a ~600K parameter count.

### B. Custom CNN Classifier

1) *Architecture*: The CNN is implemented in PyTorch and consists of four convolutional blocks followed by a two-layer MLP classifier (Table III). Each block contains two  $3 \times 3$  convolutions with batch normalization and ReLU. Blocks 1–3 use  $2 \times 2$  max-pooling; block 4 uses adaptive average pooling to produce a  $1 \times 1$  feature map regardless of input size.

TABLE III: CNN Architecture Summary

| Layer           | Out Channels | Spatial Size     | Pooling              |
|-----------------|--------------|------------------|----------------------|
| Input           | 3            | $160 \times 160$ | –                    |
| Conv Block 1    | 32           | $80 \times 80$   | MaxPool $2 \times 2$ |
| Conv Block 2    | 64           | $40 \times 40$   | MaxPool $2 \times 2$ |
| Conv Block 3    | 128          | $20 \times 20$   | MaxPool $2 \times 2$ |
| Conv Block 4    | 192          | $1 \times 1$     | AdaptiveAvgPool      |
| Flatten         | 192          | –                | –                    |
| Dropout (0.35)  | –            | –                | –                    |
| Linear          | 128          | –                | –                    |
| BatchNorm1d     | 128          | –                | –                    |
| ReLU            | –            | –                | –                    |
| Dropout (0.175) | –            | –                | –                    |
| Linear          | $N$          | –                | –                    |

where  $N = 6$  is the number of enrolled identities. All convolutional kernels are  $3 \times 3$  with padding=1 to preserve spatial dimensions before pooling. Batch normalization follows every convolution.

2) *Training Procedure*: Hyperparameters are listed in Table IV. Images are normalized using ImageNet statistics (mean = (0.485, 0.456, 0.406), std = (0.229, 0.224, 0.225)). Label smoothing of 0.05 prevents over-confident softmax outputs, which would otherwise compress the probability gap between top predictions and impair threshold calibration.

TABLE IV: CNN Training Hyperparameters

| Hyperparameter          | Value                 |
|-------------------------|-----------------------|
| Input resolution        | $160 \times 160$ px   |
| Batch size              | 16                    |
| Optimizer               | AdamW                 |
| Learning rate           | $3 \times 10^{-4}$    |
| Weight decay            | $1 \times 10^{-4}$    |
| Loss function           | CrossEntropyLoss      |
| Label smoothing         | 0.05                  |
| LR scheduler            | ReduceLROnPlateau     |
| Scheduler patience      | 3 epochs (factor 0.5) |
| Early stopping patience | 8 epochs              |
| Max epochs              | 35                    |
| Dropout (block output)  | 0.35 / 0.175          |
| Random seed             | 42                    |

The best model checkpoint (lowest validation loss) is saved automatically. Training is stopped when validation loss fails to improve for 8 consecutive epochs.

### C. InsightFace Embedding Model

The InsightFace module uses the pre-trained buffalo\_1 model pack, which bundles RetinaFace for detection (at  $640 \times 640$  resolution) and a ResNet-based ArcFace encoder producing  $L_2$ -normalized **512-dimensional embeddings**.

1) *Why InsightFace?*: The custom CNN requires offline retraining whenever a new identity is added. InsightFace’s pre-trained embeddings generalize to unseen identities without retraining, enabling *zero-shot* enrollment: a new person is recognized the moment their embedding is added to the index.

2) *Index Construction*: After CNN training, the InsightFace index is built from the training split:

- 1) Extract 512-d embeddings for every training image of each enrolled identity.
- 2) Compute the *prototype*  $\mu_c$  as the mean embedding (re-normalized to unit length) for class  $c$ .
- 3) Store individual sample embeddings alongside prototypes.
- 4) Serialize as a compressed NumPy archive (models/face\_index.npz).

3) *Inference Scoring*: Given a query embedding  $\mathbf{q}$  (unit-normalized), the composite score for class  $c$  is:

$$s_c = \alpha \cdot (\mu_c^\top \mathbf{q}) + (1 - \alpha) \cdot \max_{i: y_i=c} (\mathbf{e}_i^\top \mathbf{q}) \quad (1)$$

where  $\mathbf{e}_i$  are stored sample embeddings and  $\alpha = 0.65$  weights stable prototypes more heavily than individual sample scores. The predicted identity is  $\arg \max_c s_c$ , and the face is *Unknown* if  $\max_c s_c < \tau$ .

## VI. EVALUATION RESULTS

### A. CNN Classifier

Table V summarizes the CNN evaluation results.

TABLE V: CNN Evaluation Results

| Split      | Accuracy | Loss   | Samples |
|------------|----------|--------|---------|
| Validation | 88.89%   | 0.4224 | 18      |
| Test       | 83.33%   | —      | 18      |

Table VI shows the per-class precision, recall, and F1-score on the validation set. Akhil and Jacob show lower recall due to their smaller support counts (3 and 2 images respectively), meaning a single misclassification has an outsized impact. One Akhil image was misclassified as HariHaran, and one Jacob image was misclassified as Akhil.

TABLE VI: Validation Classification Report (CNN)

| Class        | Prec. | Recall | F1    | Support |
|--------------|-------|--------|-------|---------|
| Akhil        | 0.667 | 0.667  | 0.667 | 3       |
| Haree        | 1.000 | 1.000  | 1.000 | 3       |
| HariHaran    | 0.750 | 1.000  | 0.857 | 3       |
| Harsha       | 1.000 | 1.000  | 1.000 | 3       |
| Jacob        | 1.000 | 0.500  | 0.667 | 2       |
| Jousha       | 1.000 | 1.000  | 1.000 | 4       |
| Accuracy     |       |        | 0.889 | 18      |
| Macro avg    | 0.903 | 0.861  | 0.865 | 18      |
| Weighted avg | 0.903 | 0.889  | 0.884 | 18      |

Table VII shows the validation confusion matrix. Two errors occur: one Akhil image predicted as HariHaran, and one Jacob image predicted as Akhil, likely caused by visual similarity or lighting variation.

TABLE VII: Validation Confusion Matrix (CNN)

| True \ Pred | Akhil | Haree | HHaran | Harsha | Jacob | Jousha |
|-------------|-------|-------|--------|--------|-------|--------|
| Akhil       | 2     | 0     | 1      | 0      | 0     | 0      |
| Haree       | 0     | 3     | 0      | 0      | 0     | 0      |
| HariHaran   | 0     | 0     | 3      | 0      | 0     | 0      |
| Harsha      | 0     | 0     | 0      | 3      | 0     | 0      |
| Jacob       | 1     | 0     | 0      | 0      | 1     | 0      |
| Jousha      | 0     | 0     | 0      | 0      | 0     | 4      |

On the test set, the CNN achieves **83.33% accuracy** across all six classes (Table VIII). Akhil (1 misclassified as HariHaran), Jacob (1 as HariHaran), and Jousha (1 as HariHaran) account for the three errors.

TABLE VIII: Test Classification Report (CNN)

| Class        | Prec. | Recall | F1    | Support |
|--------------|-------|--------|-------|---------|
| Akhil        | 1.000 | 0.667  | 0.800 | 3       |
| Haree        | 1.000 | 1.000  | 1.000 | 3       |
| HariHaran    | 0.500 | 1.000  | 0.667 | 3       |
| Harsha       | 1.000 | 1.000  | 1.000 | 3       |
| Jacob        | 1.000 | 0.500  | 0.667 | 2       |
| Jousha       | 1.000 | 0.750  | 0.857 | 4       |
| Accuracy     |       |        | 0.833 | 18      |
| Macro avg    | 0.917 | 0.819  | 0.832 | 18      |
| Weighted avg | 0.917 | 0.833  | 0.842 | 18      |

TABLE IX: Test Confusion Matrix (CNN)

| True \ Pred | Akhil | Haree | HHaran | Harsha | Jacob | Jousha |
|-------------|-------|-------|--------|--------|-------|--------|
| Akhil       | 2     | 0     | 1      | 0      | 0     | 0      |
| Haree       | 0     | 3     | 0      | 0      | 0     | 0      |
| HariHaran   | 0     | 0     | 3      | 0      | 0     | 0      |
| Harsha      | 0     | 0     | 0      | 3      | 0     | 0      |
| Jacob       | 0     | 0     | 1      | 0      | 1     | 0      |
| Jousha      | 0     | 0     | 1      | 0      | 0     | 3      |

### B. InsightFace Evaluation

The InsightFace model was evaluated on the same validation and test splits using cosine similarity against stored prototypes. Results are shown in Table X.

TABLE X: InsightFace Evaluation Results

| Metric              | Value                                              |
|---------------------|----------------------------------------------------|
| Classes enrolled    | 6 (Akhil, Haree, HariHaran, Harsha, Jacob, Jousha) |
| Validation accuracy | 94.44%                                             |
| Test accuracy       | 88.89%                                             |
| Unknown threshold   | 0.45                                               |

InsightFace achieves higher accuracy than the CNN on both splits (94.44% vs. 88.89% validation; 88.89% vs. 83.33% test), demonstrating that the pre-trained ArcFace embeddings remain highly discriminative even with only 6 enrolled identities. Unlike the CNN—which must generalize a learned classifier—InsightFace compares query embeddings directly to stored prototypes, making it naturally robust to limited training data.

### C. Model Comparison

Table XI summarizes the key trade-offs between both models.

TABLE XI: CNN vs. InsightFace Comparison

| Property          | CNN             | InsightFace        |
|-------------------|-----------------|--------------------|
| Val accuracy      | 88.89%          | 94.44%             |
| Test accuracy     | 83.33%          | 88.89%             |
| Detection         | Haar Cascade    | RetinaFace         |
| Embedding dim     | $N$ classes     | 512-d              |
| Enrollment        | Offline retrain | Background rebuild |
| Inference speed   | Fast (CPU)      | Moderate (ONNX)    |
| Unknown detection | Dual threshold  | Cosine threshold   |

## VII. UNKNOWN DETECTION AND THRESHOLD SELECTION

### A. The Open-Set Problem

A vanilla softmax classifier always assigns high probability to *some* class, even for a completely novel face. Presenting an unknown face to the CNN will always yield a “winner”—but with lower confidence and a smaller gap between the top-two predictions compared to a genuine enrolled face.

### B. CNN Dual-Threshold Decision Rule

FaceVault applies the following decision rule on every prediction:

$$\text{isKnown} = \underbrace{(p_1 \geq \tau_{\text{conf}})}_{\text{confidence check}} \wedge \underbrace{(p_1 - p_2 \geq \tau_{\text{margin}})}_{\text{margin check}} \quad (2)$$



where  $p_1$  and  $p_2$  are the top-two softmax probabilities,  $\tau_{\text{conf}} = 0.655$  is the confidence floor, and  $\tau_{\text{margin}} = 0.207$  is the recognition margin.

The *margin check* is essential: a near-tie such as  $p_1 = 0.52$ ,  $p_2 = 0.48$  passes the confidence floor but indicates genuine ambiguity and should be flagged as Unknown.

### C. Threshold Calibration

Both thresholds are calibrated from the validation split by a grid search over 91 confidence values in  $[0.45, 0.90]$ . The selected threshold maximizes balanced accuracy:

$$\text{BA} = \frac{1}{2}(\text{TPR} + \text{TNR}) \quad (3)$$

Since only enrolled identities appear in the training data, the calibration treats validation softmax scores below a synthetic floor as Unknown, simulating open-set conditions. The calibrated values  $\tau_{\text{conf}} = 0.655$  and  $\tau_{\text{margin}} = 0.207$  are persisted in `models/model_metadata.json`.

### D. Runtime Threshold Adjustment

The user can override the threshold via the dashboard slider (range  $[0.40, 0.95]$ ) without retraining. The trade-off is summarized in Table XII.

TABLE XII: Effect of Threshold on Recognition Behavior

| Threshold | Behavior                    | Typical Use               |
|-----------|-----------------------------|---------------------------|
| 0.40–0.50 | Accept low-confidence faces | Permissive / high-traffic |
| 0.655     | Balanced (trained default)  | General deployment        |
| 0.80–0.95 | Accept only high-confidence | Security-critical         |

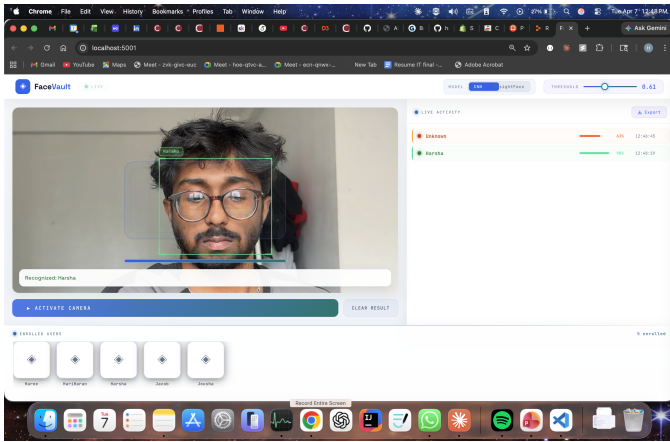


Fig. 3: Runtime threshold slider (CNN mode,  $\tau = 0.61$ ). Recognized face shown with green bounding box and name label. Lowering the threshold accepts more faces; raising it reduces false positives at the cost of more Unknown rejections.

### E. InsightFace Threshold

The InsightFace module uses a single cosine-similarity threshold  $\tau = 0.45$ , calibrated on the validation split via grid search over balanced accuracy. The prototype blend  $\alpha = 0.65$  in Eq. (1) weights stable prototypes over individual

samples, reducing sensitivity to noisy enrollment captures. The lower threshold (0.45 vs. 0.65 for the CNN) reflects the tighter cosine-similarity distributions produced by ArcFace embeddings.

## VIII. APPLICATION ARCHITECTURE

### A. System Overview

FaceVault is implemented as a Flask + Socket.IO web server running on port 5001. The system consists of four major components:

- **Camera loop thread:** Continuously reads frames from the webcam via OpenCV VideoCapture, applies face detection, stability filtering, and recognition.
- **Recognition engine** (`face_engine.py`): Abstracts over the CNN and InsightFace backends with a unified predict/enroll API.
- **SQLite activity log** (`facevault.db`): Records each unique detection event (timestamp, name, confidence, session ID).
- **Single-page dashboard:** Served as static HTML/JS/CSS with real-time Socket.IO updates.

### B. Multi-Frame Stability Consensus

Raw per-frame predictions are unreliable due to motion blur, partial occlusion, and detection jitter. FaceVault applies the following protocol before invoking the model:

- 1) Track the bounding box across consecutive frames using  $\text{IoU} \geq 0.88$  as the stability criterion.
- 2) Buffer 5 consecutive stable face crops.
- 3) Require at least 4 stable frames before recognition.
- 4) Discard frames with Laplacian variance  $< 75.0$  (blur) or face size  $< 110$  px.

### C. Live Alignment Guidance

A guide box covering 42% of the frame is overlaid on the video feed. If the detected face center deviates by more than 18% of the guide-box dimensions, a directional instruction (e.g., “Move slightly right.”) is emitted to the UI.

### D. Enrollment Flow

When a face falls below the confidence threshold:

- 1) An `unknown_detected` Socket.IO event is emitted, carrying a Base64-encoded face crop.
- 2) The “New Person” side panel opens, displaying the captured face.
- 3) The user enters a name and clicks **Add to FaceVault**.
- 4) Face crops are saved to `dataset/captured/{name}/`.
- 5) The InsightFace index is rebuilt in a background thread.
- 6) The new identity is recognizable within seconds of rebuild completion.

## E. Application Screenshots

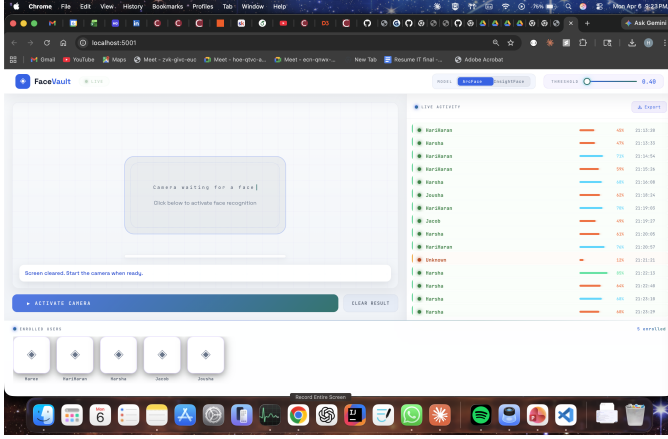
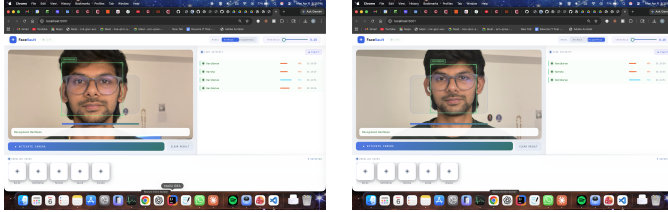


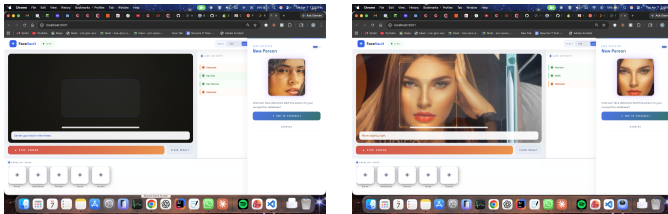
Fig. 4: FaceVault home screen. A recognized face (Harsha, 98%) is highlighted with a green bounding box. The live activity log on the right records each unique detection with name, confidence, and timestamp.



(a) CNN (ArcFace) model

(b) InsightFace model

Fig. 5: HariHaran recognized by both models. CNN reports a softmax probability; InsightFace reports cosine similarity. The model toggle (top-right) switches backends at runtime without restarting the server.



(a) Unknown face detected

(b) Name entry and enrollment

Fig. 6: Open-set detection and enrollment flow. When a face falls below the confidence threshold, the cropped face is displayed and the user is prompted to assign a name.

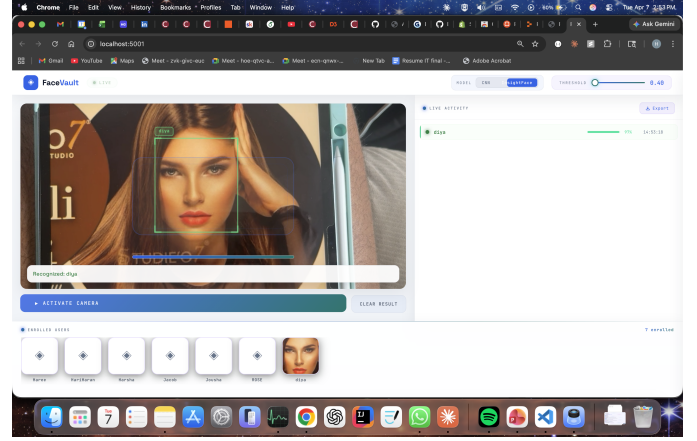


Fig. 7: Enrolled user gallery (six members) after dynamic enrollment. The newly added identity is immediately recognizable via InsightFace without restarting the server.

## F. REST API Summary

TABLE XIII: Key REST API Endpoints

| Method | Endpoint                 | Purpose                |
|--------|--------------------------|------------------------|
| POST   | /api/camera/start        | Activate camera        |
| POST   | /api/camera/stop         | Stop camera            |
| POST   | /api/threshold           | Adjust threshold       |
| POST   | /api/model/switch        | Toggle CNN/InsightFace |
| GET    | /api/known_users         | List enrolled users    |
| POST   | /api/enroll/confirm      | Confirm enrollment     |
| POST   | /api/remove_user         | Delete enrolled user   |
| GET    | /api/activity_log/export | Export CSV log         |

## IX. LIMITATIONS AND FUTURE WORK

### A. Limitations

- 1) **Small dataset:** With 16–37 raw images per identity and only 18 test samples across six classes, the reported accuracies (CNN 83.33%, InsightFace 88.89%) may not generalize to larger or more diverse populations. A rigorous evaluation would need hundreds of identities.
- 2) **Closed-set threshold calibration:** The unknown threshold is calibrated without genuine unknown faces, which may cause the system to overestimate its open-set rejection capability.
- 3) **No liveness detection:** The system cannot distinguish live faces from printed photos or digital screens. During testing, a model's face displayed on a phone screen was successfully enrolled ("diya"), demonstrating a spoofing vulnerability.
- 4) **CNN requires offline retraining for new identities:** Only the InsightFace index supports instant enrollment.
- 5) **Haar Cascade limitations:** Struggles with large pose angles ( $> 45^\circ$  yaw), heavy occlusion, and non-frontal profiles.
- 6) **Single-face pipeline:** The stability consensus is optimized for one face at a time; simultaneous multi-face recognition is not supported.

- 7) **Illumination sensitivity:** Training images are predominantly captured under similar indoor lighting; performance under low-light or back-lit conditions has not been formally evaluated.

#### B. Future Work

- 1) **Liveness detection:** Integrate passive anti-spoofing (texture analysis, optical flow) to reject photo/screen attacks.
- 2) **Larger dataset:** Collect data across multiple sessions with controlled variation in pose ( $0^\circ$ – $90^\circ$ ), lighting, and accessories.
- 3) **Continual learning:** Implement class-incremental CNN updates using experience replay or EWC [8] to avoid full retraining.
- 4) **Open-set calibration with genuine unknowns:** Include real unknown face images (e.g., from MS-Celeb-1M [9]) during threshold calibration.
- 5) **Multi-face tracking:** Extend the pipeline using SORT [10] to handle simultaneous recognition of multiple faces.
- 6) **Model compression:** Quantize and prune the CNN for deployment on edge devices (Raspberry Pi, Jetson Nano).
- 7) **Privacy controls:** Add per-user data deletion, on-device processing options, and configurable data retention policies.

#### X. CONCLUSION

This paper presented FaceVault, a dual-model real-time face recognition system combining a custom CNN classifier with InsightFace ArcFace embeddings. Trained on 178 raw images of six enrolled team members and augmented to 1406 total samples (1370 training), the CNN achieves 88.89% validation accuracy and 83.33% test accuracy across the six-class task. InsightFace achieves 94.44% validation accuracy and 88.89% test accuracy using pre-trained embeddings. A calibrated dual-threshold decision rule (confidence  $\geq 0.655$  and margin  $\geq 0.207$ ) provides robust open-set rejection. Dynamic enrollment via the web interface adds new identities in seconds through InsightFace background index rebuilding. Multi-frame stability consensus, blur detection, and alignment guidance yield reliable inference under real-world conditions. The application is fully self-contained, requiring no proprietary cloud APIs, and can be deployed on any laptop with a webcam.

#### ACKNOWLEDGMENT

The authors thank the COMP 6721 instructors at Concordia University for their guidance. The InsightFace and ArcFace research communities are acknowledged for their publicly available pre-trained models.

#### REFERENCES

- [1] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," in *Proc. IEEE/CVF CVPR*, pp. 4690–4699, 2019.
- [2] InsightFace, "InsightFace: An Open Source 2D & 3D Deep Face Analysis Library," <https://github.com/deepinsight/insightface>, 2021.
- [3] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in *Proc. IEEE/CVF CVPR*, pp. 815–823, 2015.
- [4] J. Deng, J. Guo, E. Ververas, I. Kotsia, and S. Zafeiriou, "RetinaFace: Single-Shot Multi-Level Face Localisation in the Wild," in *Proc. IEEE/CVF CVPR*, pp. 5203–5212, 2020.
- [5] P. Viola and M. J. Jones, "Robust Real-Time Face Detection," *Int. Journal of Computer Vision*, vol. 57, no. 2, pp. 137–154, 2004.
- [6] A. Bendale and T. E. Boulton, "Towards Open Set Deep Networks," in *Proc. IEEE/CVF CVPR*, pp. 1563–1572, 2016.
- [7] D. Hendrycks and K. Gimpel, "A Baseline for Detecting Misclassified and Out-of-Distribution Examples," in *Proc. ICLR*, 2017.
- [8] J. Kirkpatrick et al., "Overcoming Catastrophic Forgetting in Neural Networks," *PNAS*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [9] Y. Guo, L. Zhang, Y. Hu, X. He, and J. Gao, "MS-Celeb-1M: A Dataset and Benchmark for Large-Scale Face Recognition," in *Proc. ECCV*, pp. 87–102, 2016.
- [10] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," in *Proc. IEEE ICIP*, pp. 3464–3468, 2016.